RESTAURANT POS & MANAGEMENT SYSTEM

PRODUCT & ENGINEERING BLUEPRINT

OrderFlow POS (proposed name; subject to trademark and domain checks)

Version 1.0 | October 2026

PRD • TRD • Architecture • Database • UI/UX • Design System
API • Security • Setup • Testing • Roadmap • Q&A

Document Control

Purpose: provide a shared product and engineering specification for planning and implementing a professional restaurant POS system.

Status: proposed baseline. Requirements, technology versions, legal controls and implementation details should be validated during discovery and development.

Contents

• Executive Summary

• 1. Product Requirements Document (PRD)

• 2. Technical Requirements Document (TRD)

• 3. Database Design

• 4. UI/UX Design, Typography and Design System

• 5. State, Memory and Offline Strategy

• 6. Application Flows

• 7. API Design

• 8. Project Structure and Setup

• 9. Security Requirements

• 10. Testing and Quality Gates

• 11. Delivery Roadmap

• 12. Frequently Asked Questions (Q&A;)

• 13. Final Implementation Checklist

• Appendix A. Repository and Licensing Notes

Executive Summary

This blueprint defines a professional restaurant Point of Sale (POS) and management platform inspired by the publicly described capabilities of the NodeDR Restaurant POS repository. It establishes a proposed product scope, technical architecture, database model, UI/UX system, application flows, API contracts, security controls, testing strategy, setup guidance, and delivery roadmap.

The product connects the cashier, dining floor, kitchen, inventory, customers, and management reporting in one system. The recommended first release is a single-branch modular monolith with a clear path to multi-branch SaaS.

- Core MVP: authentication and roles, restaurant/branch setup, menu, tables, orders, kitchen display, billing, payment recording, basic inventory, sales reports, audit events, and backups.
- Suggested stack: Next.js, React, TypeScript, Tailwind CSS, shadcn/ui, TanStack Query, NestJS, PostgreSQL, Prisma, Socket.IO, Docker Compose, and automated tests.
- Primary engineering priorities: data integrity, secure authorization, payment correctness, recoverability, fast checkout, and reliable kitchen updates.

1. Product Requirements Document (PRD)

1.1 Product vision

Build a fast, reliable, offline-resilient restaurant operations platform that helps restaurants manage orders, billing, kitchen operations, inventory, staff, customers, and business performance.

1.2 Target users

Role	Primary needs
Owner	Revenue, costs, profit estimates, branch performance
Manager	Staff, menu, inventory, discounts, reports
Cashier	Fast checkout, receipts, refunds, payment records
Waiter	Tables, orders, modifiers, order status
Kitchen staff	Kitchen tickets, preparation status, priorities
Inventory manager	Purchases, suppliers, stock, wastage
Customer	QR menu, ordering, receipt

1.3 Feature priorities

Priority	Scope
P0 — MVP	Authentication, RBAC, restaurant setup, menu, tables, orders, kitchen tickets, billing, payment records, basic inventory, reports, audit logs, backups
P1 — Next release	QR ordering, CRM, loyalty, supplier purchase orders, recipe costing, attendance, advanced reporting, printer integrations
P2 — Future	Multi-branch analytics, AI demand forecasting, food-waste prediction, advanced integrations, menu recommendations

1.4 Functional requirements

ID	Module	Requirement
FR-01	Authentication	Login, logout, password reset, session management
FR-02	Organization	Restaurant, branch, tax, currency and time-zone settings
FR-03	Users	Roles, permissions and branch-scoped access
FR-04	Menu	Categories, items, variants, add-ons, prices and availability
FR-05	Tables	Capacity, occupancy, reservations and table assignment
FR-06	Orders	Dine-in, takeaway, delivery, modifiers and notes
FR-07	Kitchen	KOT creation, station routing and preparation status
FR-08	Billing	Discounts, tax, service charges, invoices and refunds
FR-09	Payments	Cash, UPI/card integrations, split payments and reconciliation
FR-10	Inventory	Ingredients, recipes, stock movements and wastage
FR-11	Procurement	Suppliers, purchase orders and receiving
FR-12	Customers	Profiles, order history and loyalty
FR-13	Reports	Sales, taxes, expenses, stock and cashier closing
FR-14	Notifications	Live order updates and operational alerts
FR-15	Administration	Audit trails, configuration, backups and exports

1.5 Business rules

1. Every order belongs to one restaurant and one branch.
2. Order items preserve snapshots of name, price, tax treatment and modifiers at sale time.
3. An order is not paid merely because a payment was initiated.
4. Refunds reference original payments and cannot exceed the refundable balance.
5. Only authorized staff may void orders, issue refunds or apply restricted discounts.
6. Stock changes are recorded as auditable movements, not silent quantity overwrites.
7. Completed financial records are corrected by reversals or adjustments rather than physical deletion.
8. Checkout and payment mutations must be idempotent against duplicate requests.
9. Organization and branch boundaries are enforced on the server.
10. Tax, rounding, invoice numbering and retention policies are configurable for the operating jurisdiction.

1.6 Non-functional acceptance targets

Concern	Initial target (to validate)
Performance	P95 under 300 ms for ordinary local API operations, excluding external services
Availability	99.5% monthly target for hosted application
Realtime updates	Normally visible within 1 second on a healthy local network
Security	No unresolved critical/high-severity findings before launch
Data integrity	No duplicate successful payment records from retries
Recovery	RPO 15 minutes and RTO 4 hours for hosted deployment
Accessibility	WCAG 2.2 AA design target

2. Technical Requirements Document (TRD)

2.1 Recommended stack

Layer	Technology	Purpose
Frontend	Next.js App Router, React, TypeScript	POS, dashboard, kitchen display, QR menu
UI	Tailwind CSS, shadcn/ui, Lucide	Reusable, accessible components
Client data	TanStack Query, React Hook Form, Zod	Server cache, forms and validation
Backend	NestJS, REST, Socket.IO	Business logic and real-time updates
Database	PostgreSQL, Prisma ORM	Transactional business data and migrations
Jobs/cache	Optional Redis and job queue	Caching, coordination and background work
API docs	OpenAPI/Swagger	API contracts and developer documentation
Testing	Jest/Vitest, Supertest, Playwright	Unit, integration and end-to-end tests
Deployment	Docker Compose; managed containers/VPS	Local development and hosting
Observability	Structured logs, OpenTelemetry-compatible metrics, error tracking	Monitoring and diagnostics

2.2 Engineering principles

- Start with a modular monolith, not many microservices. Extract services only when operational evidence justifies it.
- Use integer minor units where appropriate or exact decimal representations for financial calculations; never use floating-point arithmetic for money.
- Use database transactions for checkout, payment recording, refunds and stock consumption.
- Use idempotency keys for financial mutations and stable request IDs for troubleshooting.
- Post real-time events after database commit; use a transactional outbox when event loss would matter.
- PostgreSQL is the authoritative source for business records. Redis is an optimization, not the sole store of truth.
- Use migrations, strict TypeScript, automated tests and documented API contracts.

2.3 Architecture overview

```
Clients: POS terminal | Admin dashboard | Kitchen display | QR menu
↓ HTTPS / REST + authenticated WebSocket
Application gateway: validation | authentication | authorization | rate limits
↓
NestJS modular monolith:
Auth & Users | Organizations & Branches | Menu | Tables | Orders
Kitchen | Billing & Payments | Inventory & Recipes | Customers | Reports | Audit
↓
PostgreSQL + transactional outbox
↓
Event publisher / background jobs → Socket.IO clients / external integrations

Infrastructure: Docker | backups | monitoring | secrets | CI/CD
```

2.4 Suggested backend modules

- auth
- organizations
- branches
- users and permissions
- menu
- tables and reservations

- orders
- kitchen
- payments and refunds
- inventory and recipes
- suppliers and procurement
- customers and loyalty
- reports
- audit and administration

3. Database Design

3.1 Core entities

Table	Important fields / purpose
organizations	id, name, slug, status, created_at
branches	id, organization_id, name, timezone, currency, tax_settings
users	id, email, password_hash, status
memberships	user_id, organization_id, role_id, branch scope
roles / permissions	Role definitions and permission keys
menu_categories	id, branch_id, name, sort_order
menu_items	id, branch_id, category_id, name, price, tax category, availability
tables	id, branch_id, label, capacity, status
orders	id, branch_id, order_number, type, status, totals, timestamps
order_items	order_id, menu_item_id, name/price/tax snapshots, quantity
payments	order_id, method, amount, status, provider reference
refunds	payment_id, amount, reason, status
inventory_items	branch_id, name, unit, reorder level
stock_movements	inventory item, movement type, quantity delta, reference
recipes / recipe_items	Versioned menu recipe and required ingredients
kitchen_tickets	order_id, station_id, status, timestamps
audit_events	actor, scope, action, entity, timestamp
outbox_events	event id, type, payload, status, retry metadata

3.2 Database conventions

- Use UUID primary keys for most entities.
- Use TIMESTAMPTZ and retain the branch's configured time zone.
- Use NUMERIC/decimal-safe types for financial values.
- Add organization_id and/or branch_id where needed and enforce access scope in server queries.
- Use unique constraints for branch-scoped order numbers and provider references.
- Index common filters, for example (branch_id, created_at), (order_id, status).
- Use foreign keys, check constraints and controlled deletion policies.
- Version recipes and pricing rules where historical reproducibility matters.

3.3 Example order table (illustrative PostgreSQL)

```
CREATE TABLE orders (
id UUID PRIMARY KEY,
organization_id UUID NOT NULL,
branch_id UUID NOT NULL,
order_number BIGINT NOT NULL,
order_type VARCHAR(20) NOT NULL,
status VARCHAR(24) NOT NULL DEFAULT 'draft',
currency CHAR(3) NOT NULL DEFAULT 'INR',
subtotal NUMERIC(14,2) NOT NULL DEFAULT 0,
discount_total NUMERIC(14,2) NOT NULL DEFAULT 0,
tax_total NUMERIC(14,2) NOT NULL DEFAULT 0,
grand_total NUMERIC(14,2) NOT NULL DEFAULT 0,
created_by UUID NOT NULL,
created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
UNIQUE (branch_id, order_number),
CHECK (subtotal >= 0 AND discount_total >= 0
AND tax_total >= 0 AND grand_total >= 0),
CHECK (order_type IN ('dine_in','takeaway','delivery'))
);
CREATE INDEX idx_orders_branch_created
ON orders (branch_id, created_at DESC);
```

Production migrations must add appropriate foreign keys, tenant-isolation controls, order-state constraints, monetary validation and related tables for modifiers, payment allocations, tax components, suppliers, purchase orders, sessions, idempotency keys and outbox events.

4. UI/UX Design, Typography and Design System

4.1 Information architecture

- Sidebar: Overview, POS, Orders, Kitchen, Tables, Menu, Inventory, Purchasing, Customers, Staff, Reports, Settings.
- Top bar: branch selector, global search, date range, notifications, help and profile.
- Main area: page title, primary action, KPI cards, charts, tables, filters and pagination.
- Cashier POS uses a dedicated touch-friendly layout rather than the full admin shell.

4.2 Essential screens

Screen	Required elements
POS terminal	Category search, menu grid, modifiers, cart, table/order type, discount, payment and checkout
Kitchen display	New/preparing/ready lanes, ticket age, item notes, station filters, clear status actions
Floor/table management	Table layout, capacity, available/occupied/reserved states, active order and elapsed time
Inventory	Stock level, reorder thresholds, waste, expiry alerts, purchase orders and stock history
Analytics	Gross/net sales, order count, average order value, discounts, refunds, tax, category performance and estimated margin

4.3 UX rules

1. Cashiers can add items and find checkout without navigating multiple pages.
2. Destructive actions require suitable confirmation and permission.
3. Payment states distinguish pending, successful, failed and unknown.
4. Design loading, empty, offline, error and success states explicitly.
5. Preserve useful filters and context when navigating back.
6. Support keyboard navigation, visible focus and screen-reader labels.
7. Do not use color alone to communicate order or payment state.

4.4 Typography

Token	Size	Weight	Usage
Display	32 px	700	Dashboard highlights
H1	28 px	650	Page titles
H2	22 px	600	Section headings
H3	18 px	600	Card headings
Body	14 px	400	Standard interface
Body large	16 px	400	Instructions
Label	12 px	500	Metadata and table labels
KPI	28–36 px	650	Revenue and totals

Recommended primary typeface: Inter, with a system sans-serif fallback. Use tabular numerals for prices and times. Confirm font and glyph coverage for required currencies and languages.

4.5 Color tokens

Token	Hex	Usage
Primary navy	#172554	Navigation and headers
Primary teal	#0F766E	Primary actions and links
Success	#16A34A	Completed and paid
Warning	#D97706	Pending and low stock
Danger	#DC2626	Failed and critical
Page surface	#F8FAFC	Background
Text	#0F172A	Primary text
Muted	#64748B	Secondary text
Border	#E2E8F0	Dividers and outlines

4.6 Component standards

- Buttons: 40–44 px minimum height; primary, secondary and danger variants.
- Cards: 12 px radius, subtle border and minimal shadow.
- Inputs: persistent labels, validation and visible focus.
- Tables: 48–56 px rows, sortable columns and pagination.
- Status badges: text plus color and optional icon.
- Use a 4 px spacing scale: 4, 8, 12, 16, 24, 32, 48.
- Create reusable Button, Input, Dialog, DataTable, StatusBadge, MetricCard, OrderCard, MenuItemCard and EmptyState components.

4.7 Example CSS tokens

```
:root {
--color-primary: #0F766E;
--color-navigation: #172554;
--color-background: #F8FAFC;
--color-surface: #FFFFFF;
--color-text: #0F172A;
--color-muted: #64748B;
--color-border: #E2E8F0;
--color-success: #16A34A;
--color-warning: #D97706;
--color-danger: #DC2626;
--radius-card: 12px;
--radius-control: 8px;
--font-family: Inter, system-ui, sans-serif;
}
```

5. State, Memory and Offline Strategy

Treat UI memory, server caches, persistent records and offline synchronization as separate concerns.

Layer	Technology	Stores
UI state	React state / Zustand	Open cart, selected table, dialogs
Server cache	TanStack Query	Menu, orders, stock and reports
Session	Secure cookie or server session	Authentication/session references
Business records	PostgreSQL	Orders, payments, inventory, users and audit
Offline queue	IndexedDB if required	Queued commands and limited cached data
Events	Socket.IO, optional Redis	Live notifications and coordination
Files	Object storage	Exports, attachments and backup artifacts

5.1 Cart recovery

1. Keep the active cart in application state.
2. Persist a minimal recoverable draft locally where needed.
3. Associate the draft with branch, terminal and user.
4. Revalidate prices, taxes and availability at checkout.
5. Create the order with an idempotency key.
6. Clear the draft only after the server confirms the intended outcome.

5.2 Offline cases

- Internet outage: a local server may continue working if the LAN and server remain available.
- Terminal-to-server outage: independent order entry requires durable local storage, a unique command queue, retries, conflict handling, acknowledgments and reconciliation.
- Never silently replay financial transactions. Show unsynchronized status and reconcile uncertain payment outcomes.
- Avoid storing passwords, payment secrets or long-lived tokens in browser local storage.

5.3 Retention

- Keep business records according to legal and business retention rules.
- Cache only data needed for current workflows and expire stale caches.
- Archive old records rather than deleting financial history indiscriminately.
- Encrypt backups and test restoration.
- Document customer-data export, correction and deletion processes where legally permitted.

6. Application Flows

6.1 Login and initial setup

```
Open app → Authenticate → Load authorized organization/branch
→ Setup complete?
No → Restaurant setup wizard → branch, taxes, menu, users → Dashboard
Yes → Dashboard → POS / Orders / Inventory / Reports
```

6.2 Order-to-payment

```
Start order → Select type/table → Add items and modifiers
→ Validate on server → Submit order → Create kitchen ticket
→ Prepare → Ready/served → Calculate final bill → Initiate payment
→ Confirm or reconcile payment → Finalize checkout
→ Record stock consumption → Issue receipt and update reports
```

Operational order status and payment status must be separate. A restaurant may prepare an order while payment is pending, or require payment before preparation.

6.3 Order states

State	Meaning
draft	Order is being prepared
placed	Order accepted
confirmed	Optional separate acceptance state
preparing	Kitchen working
ready	Items ready
served	Handed to customer
completed	Operational workflow finished
cancelled	Cancelled through authorized workflow

Payment states should be independent: pending, processing, succeeded, failed, cancelled, partially_refunded and refunded, plus provider-specific states where necessary. Enforce allowed transitions server-side.

7. API Design

7.1 API conventions

- REST API base path: /api/v1.
- Document contracts with OpenAPI/Swagger.
- Validate request schemas and enforce tenant/branch authorization on every protected operation.
- Use consistent HTTP status codes and error bodies.
- Support pagination and filters on list endpoints.
- Use Idempotency-Key for checkout and other mutation endpoints where retries could duplicate actions.
- Use optimistic concurrency or version checks for simultaneous edits.
- Authorize Socket.IO connections and events; reconcile from API after reconnect.

7.2 Endpoint inventory

Module	Method + endpoint	Purpose
Auth	POST /api/v1/auth/login	Authenticate
Auth	POST /api/v1/auth/refresh	Refresh session/token
Auth	POST /api/v1/auth/logout	Revoke session
User	GET /api/v1/me	Current user and scopes
Menu	GET /api/v1/branches/:branchId/menu	Retrieve menu
Menu	POST /api/v1/menu-items	Create menu item
Tables	GET /api/v1/branches/:branchId/tables	List tables
Orders	POST /api/v1/branches/:branchId/orders	Create order
Orders	GET /api/v1/branches/:branchId/orders	Search orders
Orders	GET /api/v1/orders/:orderId	Get order
Orders	PATCH /api/v1/orders/:orderId/status	Change status
Kitchen	GET /api/v1/kitchen/tickets	Retrieve tickets
Payments	POST /api/v1/orders/:orderId/payments	Initiate/record payment
Payments	POST /api/v1/payments/:paymentId/refunds	Request refund
Inventory	GET /api/v1/branches/:branchId/inventory	List stock
Inventory	POST /api/v1/inventory/movements	Record movement
Reports	GET /api/v1/reports/sales	Sales summary
Audit	GET /api/v1/audit-events	Search audit records

7.3 Example create-order payload

```
{
"orderType": "dine_in",
"tableId": "table-uuid",
"items": [{
"menuItemId": "menu-item-uuid",
"quantity": 2,
"modifierIds": ["modifier-uuid"],
"notes": "Less spicy"
}],
"notes": "Serve together"
}
```

The client must not determine the final price or tax. The server retrieves pricing, validates modifiers, calculates totals and stores snapshots.

7.4 Example response

```
{
"data": {
"id": "order-uuid",
"orderNumber": 1024,
"status": "placed",
"currency": "INR",
"subtotal": "500.00",
"discountTotal": "0.00",
"taxTotal": "25.00",
"grandTotal": "525.00"
},
"meta": { "requestId": "request-uuid" }
}
```

Illustrative values only; tax and rounding must follow the configured jurisdiction and business rules.

7.5 Suggested real-time events

- order.created
- order.status_changed
- kitchen.ticket_created
- kitchen.ticket_updated
- payment.status_changed
- inventory.low_stock
- table.status_changed

Events should include event ID, schema version, timestamp, entity ID and authorized scope. Use an outbox where reliable delivery matters.

8. Project Structure and Setup

8.1 Suggested monorepo

```
orderflow-pos/
apps/
web/ # Next.js UI
api/ # NestJS modules
packages/
contracts/ # API schemas and contracts
ui/ # Shared components
config/
types/
prisma/
schema.prisma
migrations/
seed.ts
tests/
integration/
e2e/
security/
infra/
docker/
monitoring/
backups/
docs/
PRD.md
TRD.md
ARCHITECTURE.md
DATABASE.md
API.md
SECURITY.md
TESTING.md
RUNBOOK.md
docker-compose.yml
pnpm-workspace.yaml
turbo.json
.env.example
README.md
```

8.2 Windows prerequisites

- Git
- Docker Desktop with WSL2/Linux engine running
- Node.js LTS compatible with project dependencies
- pnpm
- VS Code or another IDE

8.3 Reference repository setup

```
git clone https://github.com/Raktim94/nodedr-restaurant-pos.git
cd nodedr-restaurant-pos
# Inspect README and installer before executing.
# If the current docs confirm the documented installer, use Git Bash/WSL2:
./install.sh
# Optional demo mode only if documented and desired:
./install.sh --demo
```

The reference repository has been documented as using http://localhost:1995 for its local installer. Verify current README and configuration before relying on the port or installer flags. Inspect install scripts before execution, especially for database initialization or destructive commands.

8.4 Proposed development scripts

```
pnpm install\npnpm dev\npnpm lint\npnpm typecheck\npnpm test\npnpm build
```

These are suggested scripts for your own implementation, not guaranteed commands in the reference repository.

8.5 Environment example

```
NODE_ENV=development
DATABASE_URL=postgresql://app:local_dev_password@localhost:5432/orderflow
APP_URL=http://localhost:3000
API_URL=http://localhost:4000
SESSION_SECRET=replace-with-a-random-secret
ACCESS_TOKEN_TTL=15m
LOG_LEVEL=debug
```

Use separate environment files or a secrets manager for real credentials. Never commit populated secrets.

9. Security Requirements

9.1 Identity and access

- Hash passwords with Argon2id or another suitable modern password-hashing algorithm.
- Use secure session cookies or a carefully implemented access/refresh-token design.
- Rotate and revoke sessions where appropriate; consider MFA for administrators.
- Enforce server-side RBAC and branch-scoped authorization for every protected operation.
- Rate-limit authentication and sensitive endpoints.
- Restrict refund, void, discount and cash-closing permissions.

9.2 Data and application protection

Threat	Mitigation
Cross-tenant data exposure	Server-side scope checks, database constraints/policies, negative tests
SQL injection	Parameterized queries and safe ORM use
XSS	Output escaping, safe rendering, restrictive CSP
CSRF	Appropriate CSRF protections for cookie-authenticated mutations
Duplicate checkout	Idempotency keys and transactional constraints
Fake payment confirmation	Server-side provider verification and signed webhook validation
Privilege escalation	Centralized authorization and negative permission tests
Audit tampering	Append-only audit design and restricted access
Data loss	Automated backups and restoration drills
Supply-chain risk	Lockfiles, dependency scanning, secret scanning and reviewed updates

9.3 Payment security

1. Create a pending payment record on the server.
2. Initiate payment through the provider.
3. Verify provider response or webhook signature.
4. Validate amount, currency, order reference and transaction state.
5. Update payment transactionally.
6. Reconcile uncertain transactions before retrying or refunding.
7. Record refunds against the original payment.

A browser redirect or client-side success screen is not proof of payment. Do not store raw card numbers, CVVs, passwords or provider secrets in logs.

9.4 India-specific validation

For an Indian deployment, validate GST invoice requirements, applicable tax rates, retention rules, privacy obligations under the Digital Personal Data Protection framework and applicable rules, and payment-provider requirements with qualified professionals. Do not hard-code legal assumptions.

10. Testing and Quality Gates

10.1 Test layers

Layer	Tools / scope
Unit	Jest/Vitest: tax, discounts, rounding, state transitions, permissions, recipe consumption
Integration	PostgreSQL + API: rollback, constraints, webhooks, stock, audit, tenant isolation
E2E	Playwright: login, order, kitchen, checkout, refund authorization, reports
Security/resilience	Access control, duplicate requests, disconnection, restart, session expiry, restore tests

10.2 Critical test cases

ID	Scenario	Expected result
TC-01	Create valid order	Order and ticket created exactly once
TC-02	Submit checkout twice	No duplicate charge or payment record
TC-03	Price changes while cart is open	Server applies defined pricing policy and recalculates
TC-04	Kitchen user calls admin endpoint	Request denied
TC-05	User accesses another branch	Denied unless explicitly authorized
TC-06	Webhook replay	No duplicate settlement
TC-07	Provider timeout	Payment remains reconcilable; no false success
TC-08	Stock receipt repeated	No duplicate stock increase
TC-09	Restart during order update	Database consistent; pending events recover
TC-10	Restore backup	Recovery targets met
TC-11	Client disconnects	UI shows connection state and recovers safely
TC-12	Refund exceeds balance	Request rejected

10.3 CI/CD quality gates

1. Formatting and lint checks.
2. Type checking.
3. Unit tests.
4. Integration tests against temporary PostgreSQL.
5. Production build.
6. Dependency and secret scans.
7. Optional E2E and container-image tests.

Before production, resolve critical authorization, payment-integrity and data-loss defects. Apply reviewed migrations, run health checks, confirm backup readiness and maintain a rollback or forward-fix plan.

11. Delivery Roadmap

Phase	Indicative timing	Deliverables
1. Foundation	Weeks 1–2	Requirements, repo, design tokens, auth, organization/branch model, migrations and CI
2. Core ordering	Weeks 3–4	Menu, tables, cart, order lifecycle, KOT and realtime kitchen updates
3. Checkout & stock	Weeks 5–6	Tax, cash payments, gateway integration, receipts, recipes and stock movements
4. Reports & hardening	Weeks 7–8	Sales analytics, audit, permissions, recovery, performance and pilot
5. Expansion	After MVP	QR ordering, loyalty, procurement, multi-branch, offline terminal sync, AI analytics

Planning estimate: about eight weeks for a tightly scoped MVP with an experienced small team. A solo developer, hardware integration, payment onboarding and production compliance may require substantially longer. This is a planning assumption, not a guarantee.

MVP success metrics

- Checkout completion time
- Order-entry error rate
- Kitchen ticket delivery success
- Stock reconciliation accuracy
- Backup restoration success
- Staff adoption and customer feedback

12. Frequently Asked Questions (Q&A)

Can I build this alone?

Yes. Start with a single-branch modular monolith and a limited MVP. Build authentication, menu, orders, kitchen, billing and basic reports before advanced modules.

Do I need Python or TensorFlow?

No, not for the core POS. TypeScript and PostgreSQL are sufficient. Python can later power a separate analytics or forecasting service.

Can it work without internet?

A local server may continue working during an internet outage if the LAN and server remain available. Independent offline terminals need local persistence and synchronization.

Can multiple restaurants share one deployment?

Yes, with tenant isolation, branch-aware authorization, per-tenant configuration, billing and automated isolation tests.

Can I accept UPI and card payments?

Yes, through supported payment-provider integrations. A UPI QR alone does not verify payment; card processing requires an appropriate provider workflow.

Can I print receipts and kitchen tickets?

Yes, but test the intended thermal printers, drivers, network connectivity and browser/desktop printing approach.

Should I use microservices?

Not initially. A modular monolith reduces deployment complexity. Split components only when evidence justifies it.

Can this become a SaaS business?

Potentially. Add tenant isolation, subscription management, onboarding, billing, backups, monitoring and support workflows, and validate demand and unit economics.

Can I copy and sell the reference repository?

The repository is described as AGPL-3.0 licensed. Commercial use may be possible subject to the license and applicable source-disclosure obligations for modified network-accessible versions. Obtain legal advice before launch.

How should I add AI?

First collect reliable order, inventory and waste data. Then introduce demand forecasting, replenishment suggestions, anomaly detection and menu-margin analysis as separately testable capabilities.

13. Final Implementation Checklist

- Product: finalize MVP scope and acceptance criteria; document order/payment state machines.
- Architecture: create monorepo, module boundaries, API contracts and design tokens.
- Database: create migrations, seed data, constraints, indexes and backup policy.
- Backend: implement authentication, branch-scoped permissions, menu, orders, kitchen, checkout and inventory.
- Frontend: implement cashier POS, KDS, management dashboard and explicit error/offline states.
- Reliability: add idempotency, audit events, transactional updates and event recovery.
- Testing: automate unit, integration, E2E, security and restore tests.
- Launch: verify payment/refund flows, pilot in a restaurant, monitor metrics and gather feedback.

Professional recommendation: build and validate five critical workflows first—staff login and branch selection; menu selection and order creation; real-time kitchen tickets; correct checkout/payment/receipt; and reliable reporting/stock updates. Expand only after those workflows are dependable.

Appendix A. Repository and Licensing Notes

Reference repository: https://github.com/Raktim94/nodedr-restaurant-pos

This document is a proposed implementation blueprint inspired by the previously discussed repository. It is not a guarantee that every feature listed is implemented in that repository. Verify current source code, documentation, dependencies, installer scripts, license file and release status before basing a production system on it.

The reference repository has been described as using the GNU AGPL-3.0 license. Review the exact license text and obtain qualified legal advice before distributing a derivative or offering a modified network service.